Two-Layer RNN: Forward Propagation, Loss Function, and Backpropagation Through Time (BPTT)

NLP Course · Homework 2 Derivation Report

Name: Yang Xiyuan Student ID: 524531910015

March 15, 2026

Contents

1 Notation and Model Architecture **2**
1.1 Model Overview . . . 2
1.2 Notation Table . . . 3

2 Forward Propagation and Loss Function of the Two-Layer RNN **4**
2.1 Embedding Layer . . . 4
2.2 Layer-1 RNN Hidden State Update . . . 4
2.3 Layer-2 RNN Hidden State Update . . . 4
2.4 Output Layer . . . 4
2.5 Loss Function . . . 5
2.6 Complete Forward Pass Summary ($T = 5$) . . . 5

3 Backpropagation Through Time (BPTT) for the Two-Layer RNN **5**
3.1 Overview . . . 5
3.2 Step 1: Output Layer Error Signal . . . 6
3.3 Step 2: Output Layer Parameter Gradients . . . 7
3.4 Step 3: Layer-2 RNN Error Signal . . . 7
3.5 Step 4: Layer-2 RNN Parameter Gradients . . . 7
3.6 Step 5: Layer-1 RNN Error Signal . . . 8
3.7 Step 6: Layer-1 RNN Parameter Gradients . . . 8
3.8 Step 7: Embedding Matrix Gradient . . . 9
3.9 Complete BPTT Algorithm Summary . . . 9
3.10 Analysis: Vanishing and Exploding Gradients . . . 10

4 Summary **10**

1 Notation and Model Architecture

1.1 Model Overview

This report studies a **Two-Layer Stacked RNN Language Model (Two-Layer Stacked RNNLM)**. The model consists of the following components:

1. **Embedding Layer**: Maps discrete tokens to dense continuous vectors.
2. **Layer-1 RNN**: Takes word embeddings as input and maintains hidden state $\boldsymbol{h}_t^{(1)}$ over time.
3. **Layer-2 RNN**: Takes Layer-1 hidden states as input, maintaining higher-level hidden state $\boldsymbol{h}_t^{(2)}$.
4. **Output Layer (Linear + Softmax)**: Maps $\boldsymbol{h}_t^{(2)}$ to a probability distribution over the vocabulary for next-word prediction.

Figure 1 shows the overall architecture of the two-layer RNN unrolled along the time axis (illustrated with $T = 3$).

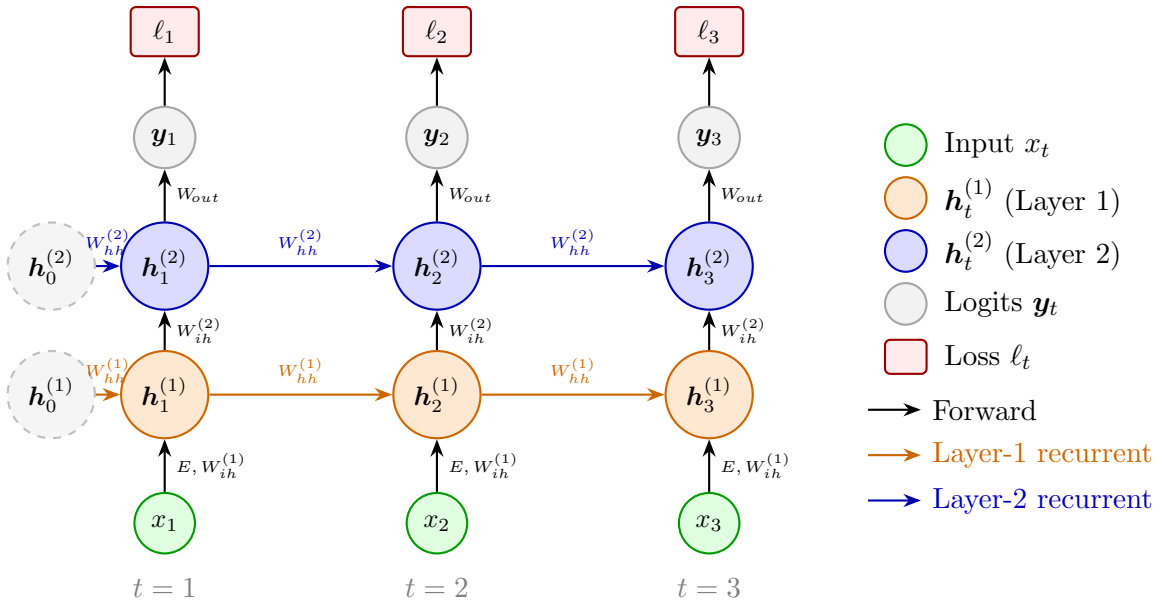

Figure 1: Two-layer stacked RNN unrolled along the time axis ($T = 3$ illustration). Green circles: input tokens; orange circles: Layer-1 hidden states; blue circles: Layer-2 hidden states; gray circles: logits; red rectangles: per-step loss. Horizontal arrows are recurrent (shared-weight) connections; vertical arrows are inter-layer and output-layer forward propagation.

1.2 Notation Table

Table 1: Summary of notation used in this report

Symbol	Dimension / Type	Meaning
T	positive integer	Sequence length (in this homework $T = 5$)
t	positive integer	Time-step index, $t \in \{1, 2, \ldots, T\}$
V	positive integer	Vocabulary size
d_e	positive integer	Word embedding dimension
d_h	positive integer	Hidden state dimension
x_t	$\mathbb{R}^V$	One-hot vector of the t-th input token
$\boldsymbol{e}_t$	$\mathbb{R}^{d_e}$	Word embedding vector at time t
$\boldsymbol{z}_t^{(\ell)}$	$\mathbb{R}^{d_h}$	Pre-activation of layer ℓ at time t
$\boldsymbol{h}_t^{(\ell)}$	$\mathbb{R}^{d_h}$	Hidden state of layer ℓ at time t ($\ell = 1, 2$)
$\boldsymbol{y}_t$	$\mathbb{R}^V$	Output logits at time t
$\hat{\boldsymbol{p}}_t$	$\mathbb{R}^V$	Predicted probability: $\hat{\boldsymbol{p}}_t = \mathrm{softmax}(\boldsymbol{y}_t)$
$\boldsymbol{o}_{t+1}$	$\mathbb{R}^V$	One-hot target vector for x_{t+1}
ℓ_t	$\mathbb{R}$	Cross-entropy loss at time t
L	$\mathbb{R}$	Total sequence loss
E	$\mathbb{R}^{d_e \times V}$	Embedding matrix
$W_{ih}^{(1)}$	$\mathbb{R}^{d_h \times d_e}$	Layer-1 input-to-hidden weight
$W_{hh}^{(1)}$	$\mathbb{R}^{d_h \times d_h}$	Layer-1 hidden-to-hidden weight
$\boldsymbol{b}^{(1)}$	$\mathbb{R}^{d_h}$	Layer-1 bias
$W_{ih}^{(2)}$	$\mathbb{R}^{d_h \times d_h}$	Layer-2 input-to-hidden weight
$W_{hh}^{(2)}$	$\mathbb{R}^{d_h \times d_h}$	Layer-2 hidden-to-hidden weight
$\boldsymbol{b}^{(2)}$	$\mathbb{R}^{d_h}$	Layer-2 bias
W_{out}	$\mathbb{R}^{V \times d_h}$	Output layer weight
$\boldsymbol{b}_{out}$	$\mathbb{R}^V$	Output layer bias
$\odot$	—	Element-wise (Hadamard) product
$(\cdot)^{\odot 2}$	—	Element-wise squaring of a vector
$\boldsymbol{\delta}_t^{(\ell)}$	$\mathbb{R}^{d_h}$	**Error signal** of layer ℓ at time t: $\partial L / \partial \boldsymbol{z}_t^{(\ell)}$
$\boldsymbol{\delta}_t^{(out)}$	$\mathbb{R}^V$	Output error signal at time t: $\partial L / \partial \boldsymbol{y}_t$

Convention. All hidden states and embedding vectors are **column vectors**; weight matrices act on column vectors from the left. For example, $W_{ih}^{(1)} \boldsymbol{e}_t$ denotes a $(d_h \times d_e)$ matrix multiplied by a d_e-dimensional column vector, yielding a d_h-dimensional column vector.

2 Forward Propagation and Loss Function of the Two-Layer RNN

2.1 Embedding Layer

The t-th input token is represented as a one-hot vector $x_t \in \mathbb{R}^V$, and is mapped to a dense continuous vector via the embedding matrix $E \in \mathbb{R}^{d_e \times V}$:

$$\boldsymbol{e}_t = E\, x_t \in \mathbb{R}^{d_e} \tag{1}$$

Since x_t is one-hot, $\boldsymbol{e}_t$ is simply the column of E indexed by the token x_t.

2.2 Layer-1 RNN Hidden State Update

The Layer-1 RNN takes $\boldsymbol{e}_t$ and the previous hidden state $\boldsymbol{h}_{t-1}^{(1)}$ as inputs, and updates the current hidden state via a linear transformation followed by tanh activation:

Layer-1 RNN Forward Pass

$$\boldsymbol{z}_t^{(1)} = W_{ih}^{(1)}\, \boldsymbol{e}_t + W_{hh}^{(1)}\, \boldsymbol{h}_{t-1}^{(1)} + \boldsymbol{b}^{(1)} \tag{2}$$

$$\boldsymbol{h}_t^{(1)} = \tanh\left(\boldsymbol{z}_t^{(1)}\right) \tag{3}$$

Boundary condition: $\boldsymbol{h}_0^{(1)} = \boldsymbol{0} \in \mathbb{R}^{d_h}$ (zero initialization).

2.3 Layer-2 RNN Hidden State Update

The Layer-2 RNN takes $\boldsymbol{h}_t^{(1)}$ as input and updates its state using its own previous hidden state $\boldsymbol{h}_{t-1}^{(2)}$:

Layer-2 RNN Forward Pass

$$\boldsymbol{z}_t^{(2)} = W_{ih}^{(2)}\, \boldsymbol{h}_t^{(1)} + W_{hh}^{(2)}\, \boldsymbol{h}_{t-1}^{(2)} + \boldsymbol{b}^{(2)} \tag{4}$$

$$\boldsymbol{h}_t^{(2)} = \tanh\left(\boldsymbol{z}_t^{(2)}\right) \tag{5}$$

Boundary condition: $\boldsymbol{h}_0^{(2)} = \boldsymbol{0} \in \mathbb{R}^{d_h}$.

2.4 Output Layer

The Layer-2 hidden state $\boldsymbol{h}_t^{(2)}$ is projected to vocabulary dimension via a linear layer, followed by Softmax to obtain the predicted probability distribution:

$$\boldsymbol{y}_t = W_{out}\, \boldsymbol{h}_t^{(2)} + \boldsymbol{b}_{out} \tag{6}$$

$$\hat{\boldsymbol{p}}_t = \mathrm{softmax}(\boldsymbol{y}_t), \quad \hat{p}_{t,k} = \frac{e^{y_{t,k}}}{\sum_{j=1}^{V} e^{y_{t,j}}} \tag{7}$$

2.5 Loss Function

The model is an autoregressive language model: at time t, it uses $x_1, \ldots, x_t$ to predict the next token x_{t+1}. The **cross-entropy loss** at each time step is:

$$\ell_t = -\log \hat{p}_{t,\,x_{t+1}} = -\log\left[\mathrm{softmax}(\boldsymbol{y}_t)\right]_{x_{t+1}} = -\boldsymbol{o}_{t+1}^{\top} \log \hat{\boldsymbol{p}}_t \tag{8}$$

For a sequence of length $T = 5$, i.e. $\vec{x} = (x_1, \ldots, x_5)$, the total loss is the mean over all time steps:

Total Loss Function ($T = 5$)

$$L = \frac{1}{T}\sum_{t=1}^{T} \ell_t = -\frac{1}{5}\sum_{t=1}^{5} \log\big[\mathrm{softmax}(\boldsymbol{y}_t)\big]_{x_{t+1}} \tag{9}$$

Minimizing L is equivalent to maximizing the log-likelihood of the sequence under the language model.

2.6 Complete Forward Pass Summary ($T = 5$)

For $t = 1, 2, 3, 4, 5$ in sequence:

$$\begin{aligned}
\boldsymbol{e}_t &= E\, x_t && \text{(Embedding)}\\
\boldsymbol{z}_t^{(1)} &= W_{ih}^{(1)}\boldsymbol{e}_t + W_{hh}^{(1)}\boldsymbol{h}_{t-1}^{(1)} + \boldsymbol{b}^{(1)}, \quad \boldsymbol{h}_t^{(1)} = \tanh(\boldsymbol{z}_t^{(1)}) && \text{(Layer-1 RNN)}\\
\boldsymbol{z}_t^{(2)} &= W_{ih}^{(2)}\boldsymbol{h}_t^{(1)} + W_{hh}^{(2)}\boldsymbol{h}_{t-1}^{(2)} + \boldsymbol{b}^{(2)}, \quad \boldsymbol{h}_t^{(2)} = \tanh(\boldsymbol{z}_t^{(2)}) && \text{(Layer-2 RNN)}\\
\boldsymbol{y}_t &= W_{out}\boldsymbol{h}_t^{(2)} + \boldsymbol{b}_{out}, \quad \hat{\boldsymbol{p}}_t = \mathrm{softmax}(\boldsymbol{y}_t) && \text{(Output Layer)}\\
L &= -\frac{1}{5}\sum_{t=1}^{5} \log \hat{p}_{t,\,x_{t+1}} && \text{(Loss)}
\end{aligned}$$

3 Backpropagation Through Time (BPTT) for the Two-Layer RNN

This section presents a detailed derivation of **Backpropagation Through Time (BPTT)** for the two-layer RNN.

3.1 Overview

BPTT unrolls the RNN along the time axis into an equivalent deep feedforward network and applies the chain rule to the unrolled graph.

For the two-layer RNN, gradients propagate in two directions:

- **Across time**: within the same layer, gradients flow backward from time step t to $t-1$ via the recurrent connections;
- **Across layers**: gradients flow from Layer 2 down to Layer 1 at the same time step t.

Figure 2 shows the BPTT computation graph (illustrated with $T = 3$), highlighting both forward and backward gradient flows.

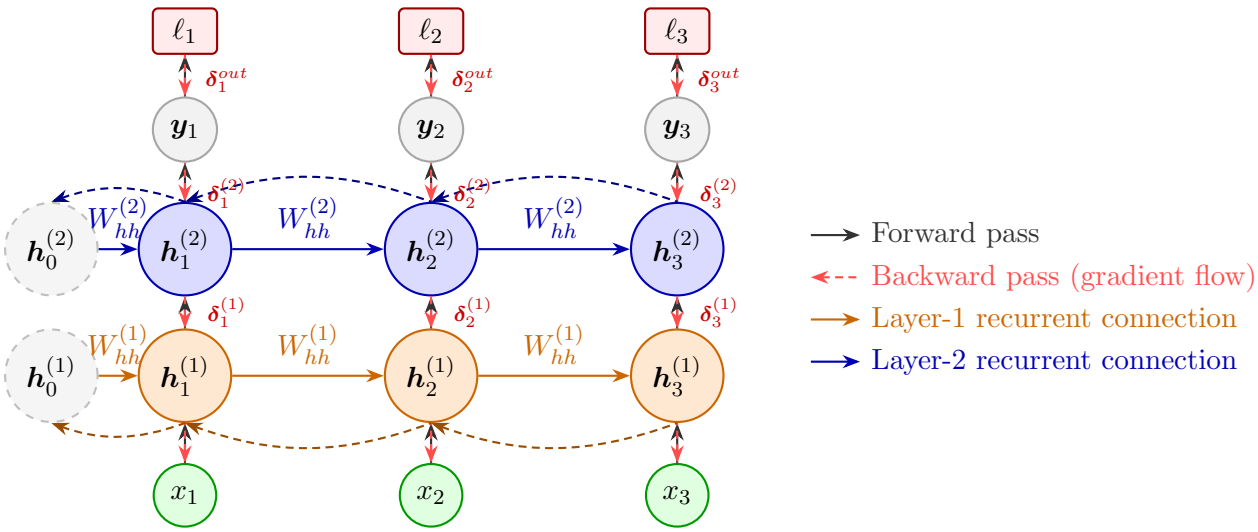

Figure 2: BPTT computation graph for the two-layer RNN ($T = 3$ illustration). Solid black arrows: forward propagation direction. Red dashed arrows: backward gradient flow, covering both cross-layer (vertical) and cross-time (horizontal) directions. Error signals $\boldsymbol{\delta}_t^{(\cdot)}$ are annotated beside the corresponding nodes.

3.2 Step 1: Output Layer Error Signal

Define the **output error signal** as the partial derivative of L with respect to the logits $\boldsymbol{y}_t$:

$$\boldsymbol{\delta}_t^{(out)} \triangleq \frac{\partial L}{\partial \boldsymbol{y}_t} \tag{10}$$

By the well-known gradient of the Softmax-CrossEntropy combination:

Step 1: Output Layer Error Signal

$$\boldsymbol{\delta}_t^{(out)} = \frac{1}{T}\left(\hat{\boldsymbol{p}}_t - \boldsymbol{o}_{t+1}\right) \tag{11}$$

Derivation. From $\ell_t = -\log \hat{p}_{t,x_{t+1}}$ and the chain rule through Softmax, for any component j:

$$\frac{\partial \hat{p}_{t,k}}{\partial y_{t,j}} = \hat{p}_{t,k}(\delta_{kj} - \hat{p}_{t,j}), \tag{12}$$

$$\frac{\partial \ell_t}{\partial y_{t,j}} = -\sum_{k=1}^{V} \frac{\delta_{k,x_{t+1}}}{\hat{p}_{t,k}} \cdot \hat{p}_{t,k}(\delta_{kj} - \hat{p}_{t,j}) = \hat{p}_{t,j} - o_{t+1,j}. \tag{13}$$

Vectorizing gives $\frac{\partial \ell_t}{\partial \boldsymbol{y}_t} = \hat{\boldsymbol{p}}_t - \boldsymbol{o}_{t+1}$, and dividing by T yields Eq. (11).

3.3 Step 2: Output Layer Parameter Gradients

From $\boldsymbol{y}_t = W_{out}\boldsymbol{h}_t^{(2)} + \boldsymbol{b}_{out}$ and the gradient formula for a matrix-vector product:

Step 2: Output Layer Parameter Gradients

$$\frac{\partial L}{\partial W_{out}} = \sum_{t=1}^{T} \boldsymbol{\delta}_t^{(out)} \left(\boldsymbol{h}_t^{(2)}\right)^\top \tag{14}$$

$$\frac{\partial L}{\partial \boldsymbol{b}_{out}} = \sum_{t=1}^{T} \boldsymbol{\delta}_t^{(out)} \tag{15}$$

Derivation. For $\boldsymbol{y} = W\boldsymbol{h} + \boldsymbol{b}$, the gradient of a scalar loss ℓ w.r.t. W is the outer product $\frac{\partial \ell}{\partial W} = \frac{\partial \ell}{\partial \boldsymbol{y}}\boldsymbol{h}^\top$, and $\frac{\partial \ell}{\partial \boldsymbol{b}} = \frac{\partial \ell}{\partial \boldsymbol{y}}$. Summing over all time steps completes the proof.

3.4 Step 3: Layer-2 RNN Error Signal

Define the Layer-2 **pre-activation error signal**:

$$\boldsymbol{\delta}_t^{(2)} \triangleq \frac{\partial L}{\partial \boldsymbol{z}_t^{(2)}} \tag{16}$$

The hidden state $\boldsymbol{h}_t^{(2)}$ affects L via two paths:

(a) **Output path**: $\boldsymbol{h}_t^{(2)} \to \boldsymbol{y}_t \to \ell_t$, contributing $W_{out}^\top \boldsymbol{\delta}_t^{(out)}$;

(b) **Temporal path**: $\boldsymbol{h}_t^{(2)} \to \boldsymbol{z}_{t+1}^{(2)} \to \cdots$ (for $t < T$), contributing $\left(W_{hh}^{(2)}\right)^\top \boldsymbol{\delta}_{t+1}^{(2)}$.

By the sum rule (with boundary condition $\boldsymbol{\delta}_{T+1}^{(2)} \triangleq \mathbf{0}$):

$$\frac{\partial L}{\partial \boldsymbol{h}_t^{(2)}} = W_{out}^\top \boldsymbol{\delta}_t^{(out)} + \left(W_{hh}^{(2)}\right)^\top \boldsymbol{\delta}_{t+1}^{(2)} \tag{17}$$

Applying the element-wise Jacobian of $\boldsymbol{h}_t^{(2)} = \tanh(\boldsymbol{z}_t^{(2)})$, namely $\operatorname{diag}\left[\mathbf{1} - (\boldsymbol{h}_t^{(2)})^{\odot 2}\right]$:

Step 3: Layer-2 Error Signal (recurrence from $t = T$ to 1)

$$\boldsymbol{\delta}_t^{(2)} = \left[W_{out}^\top \boldsymbol{\delta}_t^{(out)} + \left(W_{hh}^{(2)}\right)^\top \boldsymbol{\delta}_{t+1}^{(2)}\right] \odot \left[\mathbf{1} - \left(\boldsymbol{h}_t^{(2)}\right)^{\odot 2}\right], \quad \boldsymbol{\delta}_{T+1}^{(2)} = \mathbf{0} \tag{18}$$

Here $\mathbf{1} - (\boldsymbol{h}_t^{(2)})^{\odot 2}$ is the element-wise tanh derivative $1 - \tanh^2(\cdot)$.

3.5 Step 4: Layer-2 RNN Parameter Gradients

From $\boldsymbol{z}_t^{(2)} = W_{ih}^{(2)}\boldsymbol{h}_t^{(1)} + W_{hh}^{(2)}\boldsymbol{h}_{t-1}^{(2)} + \boldsymbol{b}^{(2)}$:

Step 4: Layer-2 Parameter Gradients

$$\frac{\partial L}{\partial W_{ih}^{(2)}} = \sum_{t=1}^{T} \boldsymbol{\delta}_t^{(2)} \left(\boldsymbol{h}_t^{(1)}\right)^\top \tag{19}$$

$$\frac{\partial L}{\partial W_{hh}^{(2)}} = \sum_{t=1}^{T} \boldsymbol{\delta}_t^{(2)} \left(\boldsymbol{h}_{t-1}^{(2)}\right)^\top \tag{20}$$

$$\frac{\partial L}{\partial \boldsymbol{b}^{(2)}} = \sum_{t=1}^{T} \boldsymbol{\delta}_t^{(2)} \tag{21}$$

Derivation. For each time step, $\boldsymbol{\delta}_t^{(2)} = \partial L / \partial \boldsymbol{z}_t^{(2)}$. The gradient of $\boldsymbol{z}_t^{(2)}$ with respect to $W_{ih}^{(2)}$ gives an outer product $\boldsymbol{\delta}_t^{(2)} (\boldsymbol{h}_t^{(1)})^\top$, similarly for $W_{hh}^{(2)}$ and $\boldsymbol{b}^{(2)}$. Summing over time steps accounts for the shared weights.

3.6 Step 5: Layer-1 RNN Error Signal

Define the Layer-1 pre-activation error signal:

$$\boldsymbol{\delta}_t^{(1)} \triangleq \frac{\partial L}{\partial \boldsymbol{z}_t^{(1)}} \tag{22}$$

The hidden state $\boldsymbol{h}_t^{(1)}$ affects L via two paths:

(a) **Upper-layer path**: $\boldsymbol{h}_t^{(1)} \to \boldsymbol{z}_t^{(2)} \to \cdots$, contributing $\left(W_{ih}^{(2)}\right)^\top \boldsymbol{\delta}_t^{(2)}$;

(b) **Temporal path**: $\boldsymbol{h}_t^{(1)} \to \boldsymbol{z}_{t+1}^{(1)} \to \cdots$ (for $t < T$), contributing $\left(W_{hh}^{(1)}\right)^\top \boldsymbol{\delta}_{t+1}^{(1)}$.

Step 5: Layer-1 Error Signal (recurrence from $t = T$ to 1)

$$\boldsymbol{\delta}_t^{(1)} = \left[\left(W_{ih}^{(2)}\right)^\top \boldsymbol{\delta}_t^{(2)} + \left(W_{hh}^{(1)}\right)^\top \boldsymbol{\delta}_{t+1}^{(1)}\right] \odot \left[\mathbf{1} - \left(\boldsymbol{h}_t^{(1)}\right)^{\odot 2}\right], \quad \boldsymbol{\delta}_{T+1}^{(1)} = \mathbf{0} \tag{23}$$

Derivation. By the chain rule, $\partial L / \partial \boldsymbol{h}_t^{(1)}$ sums contributions from both the upper-layer path (through $W_{ih}^{(2)}$) and the temporal path (through $W_{hh}^{(1)}$). Post-multiplying by the tanh Jacobian $\mathrm{diag}[\mathbf{1} - (\boldsymbol{h}_t^{(1)})^{\odot 2}]$ converts the gradient w.r.t. $\boldsymbol{h}_t^{(1)}$ into the gradient w.r.t. the pre-activation $\boldsymbol{z}_t^{(1)}$.

3.7 Step 6: Layer-1 RNN Parameter Gradients

From $\boldsymbol{z}_t^{(1)} = W_{ih}^{(1)} \boldsymbol{e}_t + W_{hh}^{(1)} \boldsymbol{h}_{t-1}^{(1)} + \boldsymbol{b}^{(1)}$:

Step 6: Layer-1 Parameter Gradients

$$\frac{\partial L}{\partial W_{ih}^{(1)}} = \sum_{t=1}^{T} \boldsymbol{\delta}_t^{(1)} \boldsymbol{e}_t^\top \tag{24}$$

$$\frac{\partial L}{\partial W_{hh}^{(1)}} = \sum_{t=1}^{T} \boldsymbol{\delta}_t^{(1)} \left(\boldsymbol{h}_{t-1}^{(1)}\right)^\top \tag{25}$$

$$\frac{\partial L}{\partial \boldsymbol{b}^{(1)}} = \sum_{t=1}^{T} \boldsymbol{\delta}_t^{(1)} \tag{26}$$

3.8 Step 7: Embedding Matrix Gradient

The gradient of L with respect to the embedding vector $\boldsymbol{e}_t$ is propagated from Layer-1:

$$\frac{\partial L}{\partial \boldsymbol{e}_t} = \left(W_{ih}^{(1)}\right)^\top \boldsymbol{\delta}_t^{(1)} \tag{27}$$

Since $\boldsymbol{e}_t = Ex_t$, the gradient of the embedding matrix E is:

Step 7: Embedding Matrix Gradient

$$\frac{\partial L}{\partial E} = \sum_{t=1}^{T} \frac{\partial L}{\partial \boldsymbol{e}_t} \cdot x_t^\top = \sum_{t=1}^{T} \left[\left(W_{ih}^{(1)}\right)^\top \boldsymbol{\delta}_t^{(1)}\right] x_t^\top \tag{28}$$

Since x_t is a one-hot vector, Eq. (28) accumulates gradients only onto the x_t-th column of E (the embedding of the t-th input token).

3.9 Complete BPTT Algorithm Summary

Complete BPTT Algorithm (Two-Layer RNNLM, T time steps)

Phase 1: Forward Pass ($t = 1 \to T$; store all intermediate quantities)

1. $\boldsymbol{e}_t = Ex_t$
2. $\boldsymbol{z}_t^{(1)} = W_{ih}^{(1)}\boldsymbol{e}_t + W_{hh}^{(1)}\boldsymbol{h}_{t-1}^{(1)} + \boldsymbol{b}^{(1)}, \;\; \boldsymbol{h}_t^{(1)} = \tanh(\boldsymbol{z}_t^{(1)})$
3. $\boldsymbol{z}_t^{(2)} = W_{ih}^{(2)}\boldsymbol{h}_t^{(1)} + W_{hh}^{(2)}\boldsymbol{h}_{t-1}^{(2)} + \boldsymbol{b}^{(2)}, \;\; \boldsymbol{h}_t^{(2)} = \tanh(\boldsymbol{z}_t^{(2)})$
4. $\boldsymbol{y}_t = W_{out}\boldsymbol{h}_t^{(2)} + \boldsymbol{b}_{out}, \;\; \hat{\boldsymbol{p}}_t = \mathrm{softmax}(\boldsymbol{y}_t)$
5. $\ell_t = -\log \hat{p}_{t,x_{t+1}}$

Phase 2: Backward Pass ($t = T \to 1$); initialize all gradients to 0; set $\boldsymbol{\delta}_{T+1}^{(1)} = \boldsymbol{\delta}_{T+1}^{(2)} = \mathbf{0}$

1. $\boldsymbol{\delta}_t^{(out)} = \frac{1}{T}(\hat{\boldsymbol{p}}_t - \boldsymbol{o}_{t+1})$
2. $\nabla W_{out} \mathrel{+}= \boldsymbol{\delta}_t^{(out)}(\boldsymbol{h}_t^{(2)})^\top, \quad \nabla \boldsymbol{b}_{out} \mathrel{+}= \boldsymbol{\delta}_t^{(out)}$
3. $\boldsymbol{\delta}_t^{(2)} = \left[W_{out}^\top \boldsymbol{\delta}_t^{(out)} + (W_{hh}^{(2)})^\top \boldsymbol{\delta}_{t+1}^{(2)}\right] \odot \left[\mathbf{1} - (\boldsymbol{h}_t^{(2)})^{\odot 2}\right]$
4. $\nabla W_{ih}^{(2)} \mathrel{+}= \boldsymbol{\delta}_t^{(2)}(\boldsymbol{h}_t^{(1)})^\top, \;\; \nabla W_{hh}^{(2)} \mathrel{+}= \boldsymbol{\delta}_t^{(2)}(\boldsymbol{h}_{t-1}^{(2)})^\top, \;\; \nabla \boldsymbol{b}^{(2)} \mathrel{+}= \boldsymbol{\delta}_t^{(2)}$

5. $\boldsymbol{\delta}_t^{(1)} = \left[(W_{ih}^{(2)})^\top \boldsymbol{\delta}_t^{(2)} + (W_{hh}^{(1)})^\top \boldsymbol{\delta}_{t+1}^{(1)}\right] \odot \left[\mathbf{1} - (\boldsymbol{h}_t^{(1)})^{\odot 2}\right]$
6. $\nabla W_{ih}^{(1)} \mathrel{+}= \boldsymbol{\delta}_t^{(1)} \boldsymbol{e}_t^\top,\ \nabla W_{hh}^{(1)} \mathrel{+}= \boldsymbol{\delta}_t^{(1)} (\boldsymbol{h}_{t-1}^{(1)})^\top,\ \nabla \boldsymbol{b}^{(1)} \mathrel{+}= \boldsymbol{\delta}_t^{(1)}$
7. $\nabla E \mathrel{+}= (W_{ih}^{(1)})^\top \boldsymbol{\delta}_t^{(1)} \cdot x_t^\top$ (only the x_t-th column is updated)

3.10 Analysis: Vanishing and Exploding Gradients

Taking Layer 2 as an example, unrolling Eq. (18) from step T back to step t yields:

$$\boldsymbol{\delta}_t^{(2)} \sim \left[\prod_{k=t+1}^{T} \operatorname{diag}\Big(\mathbf{1} - (\boldsymbol{h}_k^{(2)})^{\odot 2}\Big) \cdot \Big(W_{hh}^{(2)}\Big)^\top\right] \boldsymbol{\delta}_T^{(2)} \tag{29}$$

This reveals two well-known failure modes:

- If the spectral radius $\rho(W_{hh}^{(2)}) > 1$, the gradient grows exponentially with $(T-t)$ — **exploding gradients** — typically mitigated by **gradient clipping**;
- If $\rho(W_{hh}^{(2)}) < 1$, or if tanh saturates (derivative near zero), the gradient decays exponentially — **vanishing gradients** — making it difficult to learn long-range dependencies; **LSTM** and **GRU** are the standard remedies.

4 Summary

This report provides a complete derivation of the forward propagation and BPTT for a two-layer stacked RNNLM. The key results are:

1. **Forward Propagation** (Section 2): Information flows from the embedding layer, through Layer-1 and Layer-2 RNNs, and then through the output layer to produce a predicted distribution. Hidden states are recurrently propagated along the time axis, and the mean cross-entropy is used as the loss.
2. **Backpropagation** (Section 3): The BPTT algorithm starts from the output error signal $\boldsymbol{\delta}_t^{(out)}$ and propagates along two dimensions:
   - **Across layers**: $\boldsymbol{\delta}_t^{(out)} \to \boldsymbol{\delta}_t^{(2)} \to \boldsymbol{\delta}_t^{(1)}$ (via transposed weight matrices);
   - **Across time**: $\boldsymbol{\delta}_{t+1}^{(\ell)} \to \boldsymbol{\delta}_t^{(\ell)}$ (via $W_{hh}^{(\ell)\top}$ and the tanh derivative).

   Each parameter gradient is the sum over time steps of the outer product of the error signal and the corresponding activation.
3. **Gradient issues**: Error signals undergo T matrix multiplications in BPTT, leading to vanishing/exploding gradient risks; gradient clipping and LSTM/GRU are standard solutions.

All parameter gradients are summarized in Table 2:

Table 2: Summary of all parameter gradients for the two-layer RNNLM (∇ denotes $\partial L/\partial(\cdot)$)

Parameter	**Gradient Formula**	**Module**
W_{out}	$\sum_t \boldsymbol{\delta}_t^{(out)} (\boldsymbol{h}_t^{(2)})^\top$	Output Layer
$\boldsymbol{b}_{out}$	$\sum_t \boldsymbol{\delta}_t^{(out)}$	Output Layer
$W_{ih}^{(2)}$	$\sum_t \boldsymbol{\delta}_t^{(2)} (\boldsymbol{h}_t^{(1)})^\top$	Layer-2 RNN
$W_{hh}^{(2)}$	$\sum_t \boldsymbol{\delta}_t^{(2)} (\boldsymbol{h}_{t-1}^{(2)})^\top$	Layer-2 RNN
$\boldsymbol{b}^{(2)}$	$\sum_t \boldsymbol{\delta}_t^{(2)}$	Layer-2 RNN
$W_{ih}^{(1)}$	$\sum_t \boldsymbol{\delta}_t^{(1)} \boldsymbol{e}_t^\top$	Layer-1 RNN
$W_{hh}^{(1)}$	$\sum_t \boldsymbol{\delta}_t^{(1)} (\boldsymbol{h}_{t-1}^{(1)})^\top$	Layer-1 RNN
$\boldsymbol{b}^{(1)}$	$\sum_t \boldsymbol{\delta}_t^{(1)}$	Layer-1 RNN
E	$\sum_t \left[(W_{ih}^{(1)})^\top \boldsymbol{\delta}_t^{(1)}\right] x_t^\top$	Embedding Layer

Error signal recurrences (boundary condition: $\boldsymbol{\delta}_{T+1}^{(1)} = \boldsymbol{\delta}_{T+1}^{(2)} = \mathbf{0}$; direction: $t = T \to 1$):

$$
\begin{aligned}
\boldsymbol{\delta}_t^{(out)} &= \frac{1}{T}(\hat{\boldsymbol{p}}_t - \boldsymbol{o}_{t+1}) \\
\boldsymbol{\delta}_t^{(2)} &= \left[W_{out}^\top \boldsymbol{\delta}_t^{(out)} + (W_{hh}^{(2)})^\top \boldsymbol{\delta}_{t+1}^{(2)}\right] \odot \left[\mathbf{1} - (\boldsymbol{h}_t^{(2)})^{\odot 2}\right] \\
\boldsymbol{\delta}_t^{(1)} &= \left[(W_{ih}^{(2)})^\top \boldsymbol{\delta}_t^{(2)} + (W_{hh}^{(1)})^\top \boldsymbol{\delta}_{t+1}^{(1)}\right] \odot \left[\mathbf{1} - (\boldsymbol{h}_t^{(1)})^{\odot 2}\right]
\end{aligned}
$$